

Introduction

Single-cell RNA-seq clustering groups cells into putative cell types or transcriptional states based on transcriptome similarity. The dominant approach is graph-based community detection: construct a shared-nearest-neighbour (SNN) graph of cells in PCA space, then apply the Louvain or Leiden algorithm to identify densely connected communities. The approach scales to millions of cells, produces interpretable cluster boundaries, and is the standard in both Seurat and Bioconductor scRNA-seq workflows.

Prerequisites

A working understanding of normalised scRNA-seq counts, dimensionality reduction (PCA), and the concept of community detection in graphs.

Theory

The standard pipeline:

1. **K-nearest-neighbour graph** in reduced PCA space: each cell is connected to its k nearest neighbours (typically $k = 20$).
2. **Shared-nearest-neighbour (SNN) graph**: edge weights between cell pairs are determined by the number of shared nearest neighbours, emphasising robust local structure over noisy individual neighbour relationships.
3. **Community detection**: Louvain optimises modularity; Leiden refines Louvain to guarantee well-connected communities and has become the modern default.

The **resolution** parameter controls granularity: higher resolution yields more, smaller clusters; lower resolution yields fewer, larger ones. There is no universally correct resolution — the choice depends on the biological question and is typically validated by marker-gene analysis.

Assumptions

Cell types or states form reasonably discrete communities in PCA space; the SNN graph captures local biological structure; the chosen number of PCs (typically 10–50) captures the major biological variation.

R Implementation

```
library(Seurat)

set.seed(2026)
counts <- matrix(rpois(300 * 200, lambda = 2), 200, 300)
rownames(counts) <- paste0("g", 1:200)
colnames(counts) <- paste0("c", 1:300)

so <- CreateSeuratObject(counts)
so <- NormalizeData(so, verbose = FALSE)
so <- FindVariableFeatures(so, nfeatures = 100, verbose = FALSE)
so <- ScaleData(so, verbose = FALSE)
so <- RunPCA(so, npcs = 10, verbose = FALSE)
so <- FindNeighbors(so, dims = 1:10, verbose = FALSE)

so <- FindClusters(so, resolution = 0.3, verbose = FALSE)
n_low <- nlevels(Idsents(so))

so <- FindClusters(so, resolution = 1.2, verbose = FALSE)
n_high <- nlevels(Idsents(so))
```

```
c(low_res = n_low, high_res = n_high)
```

Output & Results

`FindClusters()` returns the same `Seurat` object with cluster identities added to the active ids and to per-cell metadata. Different resolutions produce different cluster counts; the `clustree` package visualises how cells move between clusters as resolution changes.

Interpretation

A reporting sentence: “Leiden clustering on the SNN graph (`algorithm = 4`) at resolution 0.5 produced 12 clusters spanning the major immune cell-type families; clusters were annotated by canonical marker genes (CD3 for T cells, CD19 for B cells, CD14 for monocytes) before downstream differential-expression analysis.” Always describe the algorithm, resolution, and PCs used.

Practical Tips

- Explore multiple resolutions and visualise cluster relationships across resolutions with `clustree::clustree()`; cluster stability across nearby resolutions is reassuring.
- Use Leiden (`algorithm = 4` in `Seurat` or `cluster_leiden()` in `igraph`) over Louvain — Leiden guarantees connected communities, which Louvain does not.
- Never cluster directly on UMAP coordinates; UMAP distorts global distances and clustering on it produces unstable, geometry-dependent results. Always cluster on PCA space.
- The “correct” number of clusters is a biological judgement, not a statistical one; use marker-gene analysis to confirm whether clusters correspond to known or plausible cell types.
- For very large datasets (> 500,000 cells), use `Seurat` v5 sketch-based clustering or Bioconductor’s `bluster` package with subsampling for tractable computation.
- Compare clustering to integration-aware methods (`harmony`, `Symphony`) when batch effects span multiple datasets; clustering on uncorrected data confounds batch with biology.

R Packages Used

`Seurat::FindClusters()` for graph-based clustering with Louvain/Leiden; `bluster` for Bioconductor-style clustering with multiple algorithms; `clustree` for cross-resolution stability visualisation; `igraph::cluster_leiden()` for the underlying Leiden implementation; `harmony` for batch correction before clustering.

For Reviewers

What to look for in a paper using this method.

- Common misapplications.
- Diagnostics that should be reported but often aren’t.
- Red flags in tables and figures.
- What to verify.
- What an adequate Methods paragraph must contain.

See also — labs in this chapter

- RNA-seq with DESeq2 / edgeR
- Enrichment analysis (GSEA / ORA)
- scRNA-seq with `Seurat`